Field Hours

CSCE 765 Network Security

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Name the layer, then the defense

You place a network threat on a layer before you pick a tool. This course uses 4 layers: link, internet, transport, and application. A frame that never should have entered the switch is a link problem. A packet with a lying source address is an internet problem. A session that replays an old record is a transport or record-layer problem. A password sent to the wrong name is an application problem. One product rarely covers all 4.

You write the defense next to the layer in the same row. Filtering a source prefix does not check a name in a certificate. A certificate check does not stop a host on a switch port it should not use. You will collect 4 defenses, one habit per layer, and you will refuse a diagram that marks every layer done because 1 of them is on.

Four layers, four questions

Link: who is on the segment

Internet: is the source prefix real

Transport: is the session fresh

Application: is the name right

One symptom, one layer

1. A laptop appears on a switch port you reserved for servers. You mark that as link, not as a certificate bug.

2. A packet arrives claiming a source prefix that does not belong on that interface. You mark that as internet.

3. You do not fix either one by rotating an application password. That control sits on a different layer.

Problem 1. A packet's source prefix is wrong for the interface, and the application password is strong. Which layer failed, and does the password fix it?

Answer. The internet layer failed. A strong application password does not check the source prefix. You need a defense on the layer where the lie happened.

Problem 2. You turned on one defense and marked all 4 layers done. What is the mistake?

Answer. A single control does not cover link, internet, transport, and application. You name a defense per layer. One row being green is not the other three.

Lecture 2 · 90 minutes

A port that stays shut

On the link, you do not let a port forward traffic just because a cable fits. The mechanism is port authentication: the switch leaves the port unauthorized until the endpoint completes a check. Only then does the port join the segment. A second mechanism encrypts frames between 2 neighbors, so a tap sees ciphertext. You can require both. They do different jobs.

You bind the check to the port, not to a friendly hostname the endpoint claims. If port 12 authenticates as a printer, you do not let it talk like a laptop for the rest of the day without a new check. When the link drops, you clear the authorization. A guest network that shares the same layer-2 segment as the servers has already skipped this lecture. You would rather run 2 segments than trust a label on the wall.

Open a port only after a check

1. Leave the port unauthorized

2. Check the endpoint

3. Authorize that port

4. Clear it when the link drops

Port twelve and a printer

1. Port 12 comes up. The switch forwards nothing yet. The endpoint identifies as a printer and the account check succeeds.

2. You authorize port 12 for the printer role only. A later frame that needs a laptop role does not ride that authorization.

3. The cable drops. You clear port 12. The next link has to pass the check again, even if the hostname string matches.

Problem 1. Port 12 authenticated as a printer, then the link dropped. What is the port's state before a new check?

Answer. Unauthorized. You clear the authorization when the link drops. The old success does not stick to the port, and the hostname string is not a check.

Problem 2. You encrypt frames on the segment but any cable can join it. What did you skip?

Answer. Port authentication. Encryption hides frames from a tap. It does not decide which endpoint may join. You can require both, and one does not replace the other.

Lecture 3 · 90 minutes

Drop a source that cannot belong here

You filter packets at the edge by the prefix that interface is allowed to originate. A /24 means the first 3 numbers of the address must match the interface prefix. On an interface that owns 10.1.4.0/24, a packet from 10.1.4.20 may pass the source check. A packet from 10.1.9.20 may not, because the third number is 9, not 4. You drop it.

You do not forward it and hope a later box is stricter.

You apply the rule where a lying source is cheap. Ingress on a customer interface drops sources outside that customer's prefix. You keep an allow entry per interface, not one flat list. If interface 2 drops 40 packets in a minute, you can see the rule work. The filter does not check a certificate or a session number. It only answers the source question.

Ingress check on one interface

Read the interface prefix -> Compare the first 3 numbers -> Drop a mismatch -> Log the drop count

Prefix 10.1.4.0/24

1. Interface 2 is allowed to originate 10.1.4.0/24. A packet from 10.1.4.20 matches on 10, 1, and 4.

2. A packet from 10.1.9.20 fails on the third number. You drop it on interface 2 and you do not route it onward.

3. The counter on interface 2 records the drop. Forty such drops in one minute means the rule is firing, not that the session layer was checked.

Problem 1. Interface 2 owns 10.1.4.0/24. Which source do you drop: 10.1.4.20 or 10.1.9.20?

Answer. You drop 10.1.9.20. A /24 requires the first 3 numbers to match, and the third number is 9 instead of 4. You may pass 10.1.4.20 on this check alone.

Problem 2. The filter drops 40 mismatched packets in one minute. What question did those drops answer?

Answer. Whether the source prefix belonged on that interface. They did not check a certificate, a name, or whether a session number was fresh.

Lecture 4 · 90 minutes

What the handshake actually binds

A TLS 1.3 handshake does 3 jobs in 1 round trip for a new session: it authenticates the server, it agrees on keys, and it protects application bytes. The certificate binds a key to a name. You check that name, the chain up to an anchor you already hold, and the dates. A self-signed certificate you have never pinned is not an anchor.

You do not treat a padlock as the whole check. If you asked for pay.example and the certificate names other.example, you stop, even when the chain is valid. The handshake does not filter source prefixes. It binds this session to 1 name and 1 key. When you resume, you still use the ticket rules, not a picture of a lock icon.

Checks inside one handshake

1. Match the intended name

2. Chain to a held anchor

3. Check the certificate dates

4. Then protect the bytes

The name pay.example

1. You meant to reach pay.example. The handshake offers a certificate whose name is other.example.

2. The chain leads to an anchor you hold, and the dates are valid. The name is still wrong, so you stop.

3. You do not send the password on that session. A later handshake that actually names pay.example can proceed through the 1 round trip.

Problem 1. The certificate chains to an anchor you hold, the dates are valid, and the name is other.example. You wanted pay.example. Do you continue?

Answer. No. The name must match the name you meant to reach. A valid chain for a different name is a valid certificate for that other name, not for pay.example.

Problem 2. How many round trips does a new TLS 1.3 session use in the handshake this lecture described, and what does that number not include?

Answer. 1 round trip. That figure is the handshake shape, not a source-prefix filter and not a check of the human. You still verify name, anchor, and dates.

Lecture 5 · 90 minutes

A name you can check

DNS answers are network data, and an unsigned answer is only advice. DNSSEC adds signatures so you can check a response against a chain that ends at a root anchor you already hold. You validate the signature on the record you use. A failure is a failed lookup. You do not fall back to an unsigned answer and call the 1 check done.

Absence has a signed form too. A bare error is not proof the name is missing. A signed denial covers a range, and you check that your query sits in that range. In a toy zone of 8 names, a denial for names 3 through 5 does not cover name 7. You also refuse a signature past its expiration.

Validate one signed answer

Start from the root anchor -> Check each signature -> Cover the asked name -> Reject a stale signature

A denial that covers the wrong gap

1. The toy zone has 8 names. You asked for name 7. The response is a signed denial whose range is names 3 through 5.

2. The signature checks, and the anchor is one you hold. The range does not include 7, so this denial does not answer you.

3. You do not fall back to an unsigned error. You treat the lookup as failed until a denial or a record actually covers name 7.

Problem 1. A signed denial covers names 3 through 5 in a zone of 8. You asked for name 7. Is the name proven absent?

Answer. No. The range does not include 7. A valid signature on the wrong gap is not a denial of name 7. You refuse an unsigned fallback.

Problem 2. The signature is valid but past its expiration. What do you do?

Answer. You reject it. Expiration is part of the check. A signature that used to be good is not a reason to accept the record today.

Lecture 6 · 90 minutes

Default deny between segments

You segment the network so a compromised host cannot talk to every neighbor. The mechanism is a policy that denies by default and allows the flows you listed. The application segment may reach the database segment on port 5432 and on nothing else. A host in the application segment that tries port 22 on the database is denied. You would rather maintain 5 allow rules than run one flat segment with a password as your only control.

You write the rule in the direction it applies. Inbound from the application segment is not the same rule as inbound from the internet. You log the denies. If rule 3 denies 100 flows in an hour, you look. A segment boundary does not replace the name check in the handshake. You still fail closed when the policy service is down.

Allow list versus a flat segment

Default deny

Allow port 5432 only

Five rules you can read

Deny the rest

Flat segment

Any port to any neighbor

Password is the only gate

One host sees too much

Port 5432 and a denied 22

1. The application segment may open port 5432 on the database segment. That is allow rule 1.

2. The same host opens port 22 on the database. No allow rule matches. The default deny drops it.

3. You log the deny under rule 3's counter. The handshake inside an allowed flow is still a separate check.

Problem 1. The only allow rule from the application segment to the database is port 5432. A flow targets port 22. What happens?

Answer. You deny it. Default deny drops anything that is not on the allow list. Port 22 is not 5432, so the flow does not pass.

Problem 2. You have 5 allow rules and the policy service fails. Do you forward traffic until it returns?

Answer. No. You fail closed. A segment boundary that opens when the policy service is down is a flat segment with extra steps. Logging denies does not mean allowing them.

Lecture 7 · 90 minutes

A window of fresh sequence numbers

You reject a replay by remembering what you already accepted. You accept a sequence number only when it is greater than the high-water mark and at most 4 above it. After you accept 20, the fresh values are 21, 22, 23, and 24. A repeat of 20 fails. A jump to 25 fails because it is more than 4 above 20.

When you accept a fresh number, the mark moves to it. Accept 22 and the fresh set becomes 23, 24, 25, and 26. Old numbers stay old. The window does not check the source prefix or the certificate name. You still want those 2 checks. A receiver that forgets every old number will eventually take a replay.

Move the mark, shrink the past

1. Remember the high-water mark

2. Accept only the next 4

3. Reject repeats and jumps

4. Slide the mark forward

High-water mark at twenty

1. The highest accepted sequence is 20. The fresh values are 21, 22, 23, and 24.

2. A packet numbered 22 arrives. It is inside the window and above the mark, so you accept it. The mark is now 22.

3. A repeat of 22 arrives. It is not greater than the mark, so you drop it. Fresh values are now 23 through 26.

Problem 1. The high-water mark is 20 and the window is 4. Which of 20, 24, and 25 do you accept?

Answer. Only 24. It is greater than 20 and at most 4 above it. 20 is a repeat. 25 is more than 4 above 20, so both are rejected.

Problem 2. You accept 22 with the mark at 20. What are the four fresh sequence numbers now?

Answer. 23, 24, 25, and 26. The mark moves to 22, and you accept only numbers greater than 22 and at most 4 above it. 22 itself is no longer fresh.

Lecture 8 · 90 minutes

A flow you can now defend by layer

You can now defend a flow by naming the layer and the mechanism. A switch port stays shut until the endpoint checks out. A /24 filter compares the first 3 numbers. A handshake binds a name to a key in 1 round trip only after name, anchor, and dates check. A denial must cover the name you asked for. Between segments you deny

by default.

You can also stack those checks. You accept a sequence number only when it is above the high-water mark and at most 4 above it. A packet can pass the prefix test and still be a replay. A packet can fail the prefix test before the sequence check runs. Practice on 1 flow: write the layers, the allow rule, and the window. Say which check drops the packet.

Checks you can now stack

Port authorization on the link

Prefix filter at the edge

Name check in the handshake

Sequence window on the records

Prefix first, then the window

1. Interface 2 owns 10.0.5.0/24. The high-water mark is 30, so fresh sequences would be 31, 32, 33, and 34.

2. A packet from 10.0.9.8 with sequence 33 fails the prefix test on the third number. You drop it and you do not move the mark.

3. A packet from 10.0.5.8 with sequence 33 passes the prefix test and sits in the window. You accept it and the mark becomes 33.

Problem 1. Interface 2 owns 10.0.5.0/24, and the high-water mark is 30 with a window of 4. A packet from 10.0.9.8 has sequence 33. Which check fails, and what is the mark afterward?

Answer. The prefix check fails, because a /24 needs the first 3 numbers to match and the third number is 9, not 5. You drop the packet before the sequence window. The mark stays 30. Sequence 33 would have been fresh only if the prefix had matched.

Problem 2. A later packet from 10.0.5.8 has sequence 33, the mark is still 30, and the name in the certificate is the name you wanted. Do you accept the sequence, and what are the fresh values after you do?

Answer. Yes. The source matches the /24, 33 is greater than 30 and at most 4 above it, and the name check passed. You accept 33 and the fresh values become 34, 35, 36, and 37.